

CS3014 - Estructura de Datos Avanzados

Montículos - II

Semana 2

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

18 de agosto de 2026

¿Qué aprenderemos hoy?



¿Qué aprenderemos hoy?



► Montículo de Fibonacci

Montículo de Fibonacci

Montículo de Fibonacci — idea general



Estructura

Un **montículo de Fibonacci** es una colección de árboles con raíz (no necesariamente binomiales) enlazados en una lista circular de raíces, con un puntero al mínimo.

Montículo de Fibonacci — idea general



Estructura

Un **montículo de Fibonacci** es una colección de árboles con raíz (no necesariamente binomiales) enlazados en una lista circular de raíces, con un puntero al mínimo.

¿Por qué es tan rápido INSERT, UNION y DECREASE-KEY?

Porque no reorganizan nada de inmediato: solo agregan el nuevo nodo/árbol a la lista de raíces en $O(1)$, o cortan un nodo y lo agregan como raíz en $O(1)$. Todo el “desorden” se limpia recién en EXTRACT-MIN.

Insert y Union: casi gratis



Algoritmo 1: INSERT(H, x)

$\text{grado}(x) \leftarrow 0$; $\text{padre}(x) \leftarrow \text{nulo}$; $\text{marca}(x) \leftarrow \text{falso}$;

Agregar x como un árbol de un solo nodo a la lista de raíces de H ;

si $\text{min}(H) = \text{nulo}$ **o** $\text{llave}(x) < \text{llave}(\text{min}(H))$ **entonces**

$\text{min}(H) \leftarrow x$;

Insert y Union: casi gratis



Algoritmo 3: INSERT(H, x)

$\text{grado}(x) \leftarrow 0$; $\text{padre}(x) \leftarrow \text{nulo}$; $\text{marca}(x) \leftarrow \text{falso}$;
Agregar x como un árbol de un solo nodo a la lista de raíces de H ;
si $\text{min}(H) = \text{nulo}$ **o** $\text{llave}(x) < \text{llave}(\text{min}(H))$ **entonces**
 $\text{min}(H) \leftarrow x$;

Algoritmo 4: UNION(H_1, H_2)

Concatenar las listas de raíces de H_1 y H_2 en una sola lista circular ;
 $\text{min}(H) \leftarrow \text{mín}(\text{min}(H_1), \text{min}(H_2))$;
devolver H ;

Insert y Union: casi gratis



Algoritmo 5: INSERT(H, x)

$\text{grado}(x) \leftarrow 0$; $\text{padre}(x) \leftarrow \text{nulo}$; $\text{marca}(x) \leftarrow \text{falso}$;
Agregar x como un árbol de un solo nodo a la lista de raíces de H ;
si $\text{min}(H) = \text{nulo}$ **o** $\text{llave}(x) < \text{llave}(\text{min}(H))$ **entonces**
 $\text{min}(H) \leftarrow x$;

Algoritmo 6: UNION(H_1, H_2)

Concatenar las listas de raíces de H_1 y H_2 en una sola lista circular ;
 $\text{min}(H) \leftarrow \text{mín}(\text{min}(H_1), \text{min}(H_2))$;
devolver H ;

Ambas son $O(1)$: solo se manipulan punteros de una lista circular, sin buscar ni reorganizar nada.

Decrease Key — el corte



Idea

Si al disminuir la llave de un nodo x se viola la propiedad de heap con su padre p , se **corta** x (junto con su subárbol) y se convierte en una nueva raíz.

Decrease Key — el corte



Idea

Si al disminuir la llave de un nodo x se viola la propiedad de heap con su padre p , se **corta** x (junto con su subárbol) y se convierte en una nueva raíz.

Regla de las marcas

Cada nodo puede perder **a lo mucho un hijo** sin ser cortado de su propio padre. Si un nodo pierde un **segundo** hijo, se corta también (*corte en cascada*).

Cut y Cascading-Cut



Algoritmo 7: $CUT(H, x, p)$

Quitar x de la lista de hijos de p ; $grado(p) \leftarrow grado(p) - 1$;

Agregar x a la lista de raíces de H ; $padre(x) \leftarrow \text{nulo}$; $marca(x) \leftarrow \text{falso}$;

Cut y Cascading-Cut



Algoritmo 9: $CUT(H, x, p)$

Quitar x de la lista de hijos de p ; $\text{grado}(p) \leftarrow \text{grado}(p) - 1$;
Agregar x a la lista de raíces de H ; $\text{padre}(x) \leftarrow \text{nulo}$; $\text{marca}(x) \leftarrow \text{falso}$;

Algoritmo 10: $CASCADING-CUT(H, y)$

$z \leftarrow \text{padre}(y)$;
si $z \neq \text{nulo}$ **entonces**
 si $\text{marca}(y) = \text{falso}$ **entonces**
 $\text{marca}(y) \leftarrow \text{verdadero}$;
 en otro caso
 $CUT(H, y, z)$;
 $CASCADING-CUT(H, z)$;

Decrease-Key: el algoritmo

Algoritmo 11: DECREASE-KEY(H, x, k)

```
llave( $x$ )  $\leftarrow k$  ;  
 $p \leftarrow \text{padre}(x)$  ;  
si  $p \neq \text{nulo}$  y  $\text{llave}(x) < \text{llave}(p)$  entonces  
    | CUT( $H, x, p$ ) ;  
    | CASCADING-CUT( $H, p$ ) ;  
si  $\text{llave}(x) < \text{llave}(\min(H))$  entonces  
    |  $\min(H) \leftarrow x$  ;
```

Decrease-Key: el algoritmo

Algoritmo 12: DECREASE-KEY(H, x, k)

```
llave( $x$ )  $\leftarrow k$  ;  
 $p \leftarrow \text{padre}(x)$  ;  
si  $p \neq \text{nulo}$  y  $\text{llave}(x) < \text{llave}(p)$  entonces  
    |  $\text{CUT}(H, x, p)$  ;  
    |  $\text{CASCADING-CUT}(H, p)$  ;  
si  $\text{llave}(x) < \text{llave}(\min(H))$  entonces  
    |  $\min(H) \leftarrow x$  ;
```

Igual que en la diapositiva anterior: cada corte real es $O(1)$; el análisis amortizado (más adelante) muestra que el *total* también se comporta como $O(1)$.

Corte en cascada — ejemplo



Corte en cascada — ejemplo



P estaba **marcado** (ya había perdido un hijo antes). Al cortar C , P pierde su *segundo* hijo: se corta P también y se agrega, junto con C , a la lista de raíces (sin marca).

Corte en cascada — ejemplo



P estaba **marcado** (ya había perdido un hijo antes). Al cortar C , P pierde su *segundo* hijo: se corta P también y se agrega, junto con C , a la lista de raíces (sin marca).

Amortizado: cada corte le “cobra” a la operación que marcó el nodo, no a la que lo corta — por eso DECREASE-KEY es $\Theta(1)$ amortizado.

Extract Min — remover y consolidar



Paso 1: remover

Se quita la raíz mínima; todos sus hijos pasan a la lista de raíces (sin marca, pues las raíces nunca están marcadas).

Extract Min — remover y consolidar



Paso 1: remover

Se quita la raíz mínima; todos sus hijos pasan a la lista de raíces (sin marca, pues las raíces nunca están marcadas).

Paso 2: consolidar

Se recorre la lista de raíces **enlazando** pares de árboles del mismo grado (la raíz con llave mayor se vuelve hijo de la otra), repitiendo hasta que todos los grados en la lista de raíces sean distintos — igual que un montículo binomial.

Extract-Min: el algoritmo



Algoritmo 13: EXTRACT-MIN(H)

```
 $z \leftarrow \min(H) ;$   
si  $z \neq \text{nulo}$  entonces  
    Agregar cada hijo de  $z$  a la lista de raíces, quitándole el padre ;  
    Quitar  $z$  de la lista de raíces ;  
    si  $z$  era la única raíz entonces  
         $\min(H) \leftarrow \text{nulo} ;$   
    en otro caso  
         $\min(H) \leftarrow$  alguna raíz restante ;  
        CONSOLIDATE( $H$ ) ;  
devolver  $z ;$ 
```

Consolidate: unificando grados



Algoritmo 14: CONSOLIDATE(H)

Crear arreglo $A[0..D(n)]$, todo en nulo ;
para *cada raíz w de la lista original de raíces* **hacer**

```
   $x \leftarrow w$ ;  $d \leftarrow \text{grado}(x)$  ;  
  mientras  $A[d] \neq \text{nulo}$  hacer  
     $y \leftarrow A[d]$  ;  
    si  $\text{llave}(x) > \text{llave}(y)$  entonces  
       $\perp$  intercambiar  $x$  y  $y$   
      BINOMIAL-LINK( $y, x$ ) ;  
       $A[d] \leftarrow \text{nulo}$ ;  $d \leftarrow d + 1$  ;  
   $A[d] \leftarrow x$  ;
```

Reconstruir la lista de raíces a partir de A ; actualizar $\min(H)$;

Consolidate: unificando grados



Complejidad

Consolidar cuesta $O(D(n) + t)$, donde t es el número de raíces antes de consolidar y $D(n)$ es el grado máximo posible; en amortizado, el costo de las raíces extra se “paga” con el potencial que dejaron acumulado las llamadas a INSERT y DECREASE-KEY anteriores, dando $O(\lg n)$ amortizado.

¿Por qué el grado máximo es $O(\lg n)$?



Regla (recordatorio)

Un nodo puede perder a lo mucho un hijo antes de ser cortado de su propio padre.

¿Por qué el grado máximo es $O(\lg n)$?



Regla (recordatorio)

Un nodo puede perder a lo mucho un hijo antes de ser cortado de su propio padre.

Teorema

El número mínimo de nodos en un árbol cuya raíz tiene grado k es F_{k+2} , el $(k + 2)$ -ésimo número de Fibonacci.

Idea de la demostración



Ordenando a los hijos

Sea x un nodo de grado k , y sean $y_1, \dots, y_k$ sus hijos en el orden en que fueron enlazados (del más antiguo al más reciente).

Idea de la demostración



Ordenando a los hijos

Sea x un nodo de grado k , y sean $y_1, \dots, y_k$ sus hijos en el orden en que fueron enlazados (del más antiguo al más reciente).

¿Qué grado mínimo tenía y_i al momento de enlazarse?

Cuando y_i se enlazó como el i -ésimo hijo de x , x ya tenía $i - 1$ hijos, así que (por cómo consolida EXTRACT-MIN) y_i tenía grado **al menos** $i - 2$ en ese momento.

Idea de la demostración



Ordenando a los hijos

Sea x un nodo de grado k , y sean $y_1, \dots, y_k$ sus hijos en el orden en que fueron enlazados (del más antiguo al más reciente).

¿Qué grado mínimo tenía y_i al momento de enlazarse?

Cuando y_i se enlazó como el i -ésimo hijo de x , x ya tenía $i - 1$ hijos, así que (por cómo consolida EXTRACT-MIN) y_i tenía grado **al menos** $i - 2$ en ese momento.

¿Y después?

Por la regla de las marcas, y_i puede haber perdido a lo mucho **un** hijo desde entonces. Su grado actual es, entonces, al menos $i - 2$.

Planteando la recurrencia



Definiendo s_k

Sea s_k el número mínimo de nodos posible en un árbol cuya raíz tiene grado k . Contando la raíz, el hijo más viejo (que aporta al menos s_{k-2} nodos, por el argumento anterior) y los demás hijos (cada uno con grado al menos $i - 2$, de manera creciente):

$$s_k \geq 2 + \sum_{i=2}^k s_{i-2} = 2 + \sum_{i=0}^{k-2} s_i.$$

Cerrando la recurrencia



Y esa es, precisamente, la recurrencia de Fibonacci

Se puede probar por inducción que $s_k \geq F_{k+2}$ para todo $k \geq 0$, con $F_0 = 0$, $F_1 = 1$.

Cerrando la recurrencia



Y esa es, precisamente, la recurrencia de Fibonacci

Se puede probar por inducción que $s_k \geq F_{k+2}$ para todo $k \geq 0$, con $F_0 = 0$, $F_1 = 1$.

Conclusión

Como $F_{k+2} = \Theta(\varphi^k)$ crece exponencialmente ($\varphi = \frac{1+\sqrt{5}}{2}$), un árbol con n nodos tiene grado $k = O(\lg n)$. Eso acota $D(n)$ en la complejidad de EXTRACT-MIN.

El método del potencial



La idea

Definimos una función de **potencial** Φ sobre el estado de la estructura. El **costo amortizado** de la operación i se define como

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}),$$

donde c_i es el costo real. Si Φ nunca cae por debajo de su valor inicial, la suma de costos amortizados **acota por arriba** la suma de costos reales.

El método del potencial



La idea

Definimos una función de **potencial** Φ sobre el estado de la estructura. El **costo amortizado** de la operación i se define como

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}),$$

donde c_i es el costo real. Si Φ nunca cae por debajo de su valor inicial, la suma de costos amortizados **acota por arriba** la suma de costos reales.

Potencial de un montículo de Fibonacci

$$\Phi(H) = t(H) + 2m(H),$$

donde $t(H)$ es el número de árboles en la lista de raíces, y $m(H)$ el número de nodos marcados.

Insert y Union: amortizado $O(1)$



INSERT

Costo real $O(1)$ (agregar un árbol de un nodo, sin marca, a la lista de raíces).
Cambio de potencial: $\Delta t = +1, \Delta m = 0 \Rightarrow \Delta \Phi = +1$.

$$\hat{c} = O(1) + 1 = O(1).$$

Insert y Union: amortizado $O(1)$



INSERT

Costo real $O(1)$ (agregar un árbol de un nodo, sin marca, a la lista de raíces).
Cambio de potencial: $\Delta t = +1, \Delta m = 0 \Rightarrow \Delta \Phi = +1$.

$$\hat{c} = O(1) + 1 = O(1).$$

UNION

Costo real $O(1)$ (empalmar dos listas circulares de raíces y actualizar el mínimo). $\Delta \Phi = 0$.

$$\hat{c} = O(1).$$

Decrease-Key: costo real y cambio de potencial



Costo real

Si DECREASE-KEY produce c cortes en total (el corte de x , más $c - 1$ cortes en cascada), el costo real es $c_i = O(c)$.

Decrease-Key: costo real y cambio de potencial



Costo real

Si DECREASE-KEY produce c cortes en total (el corte de x , más $c - 1$ cortes en cascada), el costo real es $c_i = O(c)$.

Cambio de potencial

- ▶ Cada uno de los c nodos cortados se vuelve una nueva raíz sin marca: $\Delta t = +c$.
- ▶ Los primeros $c - 1$ cortes **desmarcan** un nodo que ya estaba marcado (-2 cada uno); el último nodo de la cascada puede quedar recién marcado ($+2$).

$$\Delta\Phi \leq c + (-2(c - 1) + 2) = 4 - c.$$

Decrease-Key: por qué el costo amortizado es $O(1)$



El costo amortizado

$$\hat{c}_i = O(c) + (4 - c) = O(1).$$

¡El término c se **cancela**! No importa cuántos cortes en cascada ocurran: el costo amortizado siempre es constante.

Extract-Min: costo real y cambio de potencial



Costo real

Consolidar cuesta $O(D(n) + t(H))$: se examinan los $t(H)$ árboles de la lista de raíces, más los a lo mucho $D(n)$ hijos que quedaron expuestos al quitar la raíz mínima.

Extract-Min: costo real y cambio de potencial



Costo real

Consolidar cuesta $O(D(n) + t(H))$: se examinan los $t(H)$ árboles de la lista de raíces, más los a lo mucho $D(n)$ hijos que quedaron expuestos al quitar la raíz mínima.

Cambio de potencial

Después de consolidar, ningún par de raíces comparte grado, así que quedan a lo mucho $D(n) + 1$ árboles:

$$\Delta\Phi \leq (D(n) + 1) - t(H).$$

Extract-Min: por qué el costo amortizado es $O(\lg n)$



El costo amortizado

$$\hat{c}_i = O(D(n) + t(H)) + (D(n) + 1 - t(H)) = O(D(n)).$$

Extract-Min: por qué el costo amortizado es $O(\lg n)$



El costo amortizado

$$\hat{c}_i = O(D(n) + t(H)) + (D(n) + 1 - t(H)) = O(D(n)).$$

Aquí es donde usamos el teorema que acabamos de probar

Como $D(n) = O(\lg n)$ (la cota de grado, vía números de Fibonacci), concluimos: **EXTRACT-MIN** = $O(\lg n)$ **amortizado**.

Resumen de la sesión



- ▶ El **montículo de Fibonacci** es perezoso: INSERT, UNION y DECREASE-KEY son $\Theta(1)$ amortizado, y todo el trabajo se paga recién en EXTRACT-MIN, gracias a la regla de que un nodo pierde a lo mucho un hijo antes de ser cortado.
- ▶ El **método del potencial** ($\Phi = t + 2m$) es lo que formaliza ese “pago diferido”: por eso DECREASE-KEY es $O(1)$ amortizado sin importar cuántos cortes en cascada ocurran, y EXTRACT-MIN es $O(\lg n)$ amortizado gracias a la cota de grado que acabamos de probar.

Referencias



Bibliografía citada

- ▶ Cormen, Thomas H. et al. (2009). *Introduction to Algorithms*. 3rd ed. The MIT Press.
- ▶ Cormen, Thomas H. et al. (2022). *Introduction to Algorithms*. 4th ed. The MIT Press.

Agradecimientos



Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso, así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz (<https://web.stanford.edu/class/archive/cs/cs166/cs166.1256/>).

Gracias